

Requirements Mapping

Doctor Clinic DevOps System

Institute Requirements Mapping

Muhammed Munir Al Tawil · DevOps Course · May 2026

1. Purpose

This document maps the DataScientest institute validation requirements to the concrete deliverables implemented in this project. Each of the 6 required steps is covered with specific evidence from the repository and live deployment.

2. Coverage Matrix

Step	Requirement	Status	Evidence
1	Specifications	✓ Done	PDF/DOCX specification document, architecture diagram, organized repository with README
2	Infrastructure	✓ Done	Dockerfile, docker-compose.yml, Nginx config, Kubernetes manifests (10 files), backup script, live Proxmox VM
3	Data Management	✓ Done	MySQL database, 16 migrations, 9 seeders, backup script, Laravel log storage, access rights via Docker env
4	CI/CD Pipeline	✓ Done	GitLab CI 5-stage pipeline: validate → build → test → package → deploy, automated PHPUnit tests
5	Supervision	✓ Done	Monitoring plan document, docker compose ps verification, live container health checks, log access
6	Automation / IaC	✓ Done	Terraform starter (5 files), reusable YAML templates, environment variables for sensitive data

3. Detailed Evidence Per Requirement

Step 1 – Specifications

- Specifications document in PDF/DOCX format
- Business need and functional objectives defined
- Implementation architecture diagram
- GitLab repository: gitlab.com/MunirAltawil/Doctor-Clinic-DevOps-System
- Professional README with installation procedure

Step 2 – Infrastructure

- Dockerfile (PHP 8.2-FPM base, Composer, extensions)
- docker-compose.yml with 3 services, volumes, and network
- Nginx reverse proxy configuration (docker/nginx/default.conf)
- 10 Kubernetes YAML manifests in /k8s/
- Live deployment on Proxmox VE 8.4.14 – VM 100 – Ubuntu 22.04
- Public access: <http://51.158.200.127:8080>
- Two-tier network: vmbr0 (public) + vmbr1 (internal NAT)
- Disk resize: growpart + resize2fs (2.2GB → 22GB)

Step 3 – Data Management

- MySQL 5.7 container with persistent Docker volume
- 16 Laravel migrations executed successfully
- 9 database seeders with demo clinical data
- Backup automation script in /scripts/
- Laravel log storage in storage/logs/
- Environment variables for DB credentials (.env)

Step 4 – CI/CD Pipeline

- GitLab CI pipeline: 5 stages (validate, build, test, package, deploy)
- PHP syntax validation via `php -l` on all source files
- Frontend build: `npm ci + npm run build` (Vite)
- Automated PHPUnit tests with SQLite in-memory database
- Package stage creates tar.gz release artifact
- Deploy stage placeholder ready for production target

Step 5 – Supervision

- Monitoring plan documentation (monitoring-plan.md)
- Docker container health verification via `docker compose ps`
- Container log access via `docker compose logs`
- Live application health verified at <http://51.158.200.127:8080>
- Architecture prepared for future Prometheus + Grafana integration

Step 6 – Automation / IaC

- Terraform starter files in /terraform/ (provider, main, variables, outputs)

- Declarative YAML templates for Kubernetes deployment
- Environment variables used for all sensitive data
- Cloud-Init for automated VM configuration on Proxmox
- iptables-persistent for persistent NAT rules across reboots

4. Final Assessment

All 6 institute requirements are fully covered. The project goes beyond the minimum requirement by delivering a live, publicly accessible deployment on Proxmox VE in addition to the standard Docker Compose, CI/CD, Kubernetes, and IaC deliverables.

5. Repository Locations

Platform	URL
GitLab	https://gitlab.com/MunirAltawil/Doctor-Clinic-DevOps-System
Live Application	http://51.158.200.127:8080